Document Number: DXXXXR1
Date: 2020-06-15
Author: Anthony Williams
Just Software Solutions Ltd
Audience: EWG

DXXXX: Zap the Zap: Pointers should just be bags of bits

This paper relates to P1726: Pointer lifetime-end zap and provenance, too. My argument is that in many ways pointers are **already** treated like bags of bits by the language, so we should be consistent, and treat them as such throughout. A consequence of this is that there can be no "lifetime-end pointer zap".

This paper provides a series of examples. I believe all these examples are clearly defined by the standard due to the fact that pointers are *scalar types* and *trivially copyable* types.

As shown by these examples, the "pointer zap" from the final sentence of [basic.stc] p4 (“ Any other use of an invalid pointer value has implementation-defined behavior”), and especially note 31 (“Some implementations might define that copying an invalid pointer value causes a system-generated runtime fault.”) is clearly incompatible with pointers being *trivially copyable* types from [basic.types] p3. Consequently we should either strike that permission from the standard and require that invalid pointer values are still copyable and comparable, or we should decide that pointers are not *trivially copyable* after all, which would have far reachging consequences.

All standard references are to the C++ working draft from the 2020-04 mailing: N4861.

All these examples have been tested with gcc, clang and MSVC. Links to compiler explorer are provided for each example.

Wording

Strike the final sentence and note 31 from [basic.stc] p4:

> When the end of the duration of a region of storage is reached, the values of all pointers representing the address of any part of that region of storage become invalid pointer values (6.8.2). Indirection through an invalid pointer value and passing an invalid pointer value to a deallocation function have undefined behavior. ~~Any other use of an invalid pointer value has implementation-defined behavior.31~~

Add a new sentence to the end of [basic.stc] p4:

> Copying and assigning invalid pointer values preserves the value representation. Comparisons involving an invalid pointer value return an unspecified result. An invalid pointer value will become a valid pointer value if region of storage with dynamic storage duration is allocated and the value representation of a pointer to the newly allocated storage cast to the same pointer type as the erstwhile-invalid pointer value is the same as the value representation of the erstwhile-invalid pointer value.

Examples

Example 1: `memcpy` on a pointer

Compiler explorer link

```
#include <assert.h>
#include <string.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    memcpy(&y,&x,sizeof(x));
    assert(x == y);
    assert(*y==42);
}
```

Here, we use `memcpy` to copy the bits of a pointer from one pointer to another. The second pointer is now valid and points to the same thing the original did because pointers are *trivially copyable* ([basic.types] p3).

Example 2: `memcpy` via a buffer

Compiler explorer link

```
#include <assert.h>
#include <string.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));
    memcpy(&y, buffer, sizeof(x));
    assert(x == y);
    assert(*y == 42);
}
```

Here, we use `memcpy` to copy the bits of a pointer from one pointer to a buffer, and then from that buffer to another pointer. The second pointer is now valid and points to the same thing the original did because pointers are *trivially copyable* ([basic.types] p2).

Example 3: `reinterpret_cast` to an integer

Compiler explorer link

```
#include <assert.h>
#include <string.h>
#include <stdint.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    uintptr_t temp= reinterpret_cast<uintptr_t>(x);
    y= reinterpret_cast<int *>(temp);
    assert(x == y);
    assert(*y == 42);
}
```

Here we rely on the provision of [expr.reinterpret.cast] p5 that a pointer may be cast to an integer and back and retain its value.

Example 4: `memcpy` with modifications

Compiler explorer link

```
#include <assert.h>
#include <string.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));
    for(auto &c : buffer) {
        c^= 0x55;
    }
    for(auto &c : buffer) {
        c^= 0x55;
    }
    memcpy(&y, buffer, sizeof(x));
    assert(x == y);
    assert(*y == 42);
}
```

Now we take example 1 a step further: we perform a reversible modification on the bits in the buffer after the first `memcpy`, then reverse that modification and `memcpy` it back. Since the bits in the buffer now hold their original values, we can copy them to a pointer, which will have the same value, because pointers are *trivially copyable*.

Example 5: `memcpy` and write to file

Compiler explorer link

```
#include <assert.h>
#include <string.h>
#include <stdio.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));

    auto file= fopen("tempfile", "wb");
    auto written= fwrite(buffer, 1, sizeof(buffer), file);
    assert(written == sizeof(buffer));
    fclose(file);

    memset(buffer, 0, sizeof(buffer));

    file= fopen("tempfile", "rb");
    auto read= fread(buffer, 1, sizeof(buffer), file);
    assert(read == sizeof(buffer));
    fclose(file);
    memcpy(&y, buffer, sizeof(x));
    assert(x == y);
    assert(*y == 42);
```

```
}
```

This time we are copying the pointer to a buffer, writing our bytes to a file, clearing the buffer and reading the bytes back from the file, then copying the bytes back to the pointer. If our file is unmodified then the buffer will have the same contents after reading as it did before writing, so copying the buffer back to the pointer yields the same value, and the pointer is again valid and points to the same object.

Example 6: destroy and recreate the object

Compiler explorer link

```
#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>

struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));

    x->~X();
    new(x) X{99};

    memcpy(&y, buffer, sizeof(x));
    assert(x == y);
    assert(y->i == 99);
    assert(x->i == 99);
}
```

This time, we destroy the pointed-to object and recreate a new object with a new value at the same memory location.

The pointer `x` still holds the same bit pattern, and still points to a valid object, so both the original pointer `x` and the newly constructed copy `y` point to the new object, and all is well by [basic.life] p8.

Example 7: `delete` and `new` the object

Compiler explorer link

```
#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>

struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));
```

```
    delete x;
    y= new X{99};

    unsigned char buffer2[sizeof(x)];
    memcpy(buffer2, &y, sizeof(x));

    if(memcmp(buffer, buffer2, sizeof(x))) {
        printf("Different address\n");
        return 0;
    }

    memcpy(&x, buffer2, sizeof(x));

    assert(x == y);
    assert(y->i == 99);
    assert(x->i == 99);
}
```

This time, we destroy the pointed-to object with `delete` and recreate a new object with a new value with `new`.

We then copy the new pointer into a buffer and compare the buffers. If the buffers are different, then the pointers are clearly different and our test doesn't work, so we stop.

If the buffers are the same, then we copy the new buffer (which is a copy of our new pointer) into the old pointer.

`x` is now a copy of the raw bits of our new pointer, so everything must work.

Example 8: `delete` and `new` the object again

Compiler explorer link

```
#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>

struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));

    delete x;
    y= new X{99};

    unsigned char buffer2[sizeof(x)];
    memcpy(buffer2, &y, sizeof(x));

    if(memcmp(buffer, buffer2, sizeof(x))) {
        printf("Different address\n");
        return 0;
    }

    assert(x == y);
    assert(y->i == 99);
    assert(x->i == 99);
```

```
}
```

This is the same as example 7, except we don't copy the raw bits from the new buffer over our old pointer.

We know that the bits of `x` and the bits of `y` are the same because we compared them with `memcmp`. Since the pointers are trivially copyable, the value of the pointer is determined by the *value representation*, which is the set of bits of the *object representation*. Since we know the *object representation* is the same, the *value representation* must be the same, so the pointers must have the same value.

Since the pointers must have the same value, `x` must be equal to `y`, and must point to the same object, and all is well.

Example 9: using `std::atomic` to hold the pointer

Compiler explorer link

```
#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>
#include <atomic>

struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;
    std::atomic<X *> p(x);

    delete x;
    y= new X{99};

    X *temp= y;
    if(!p.compare_exchange_strong(temp, y)) {
        printf("Different address\n");
        return 0;
    }

    assert(x == y);
    assert(y->i == 99);
    assert(x->i == 99);
}
```

This is the same as example 8, except instead of using `memcmp` to determine the equivalence, we use `compare_exchange_strong`, which compares pointer as-if with `memcmp`.

Example 10: using `std::atomic` to hold the pointer, comparison the other way round

Compiler explorer link

```
#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>
#include <atomic>
```

```
struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;

    delete x;
    y= new X{99};

    std::atomic<X *> p(y);
    if(!p.compare_exchange_strong(x, nullptr)) {
        printf("Different address\n");
        return 0;
    }

    assert(x == y);
    assert(y->i == 99);
    assert(x->i == 99);
}
```

This is the same as example 9, except that rather than comparing the `temp` value copied from `y` with our stored pointer, we store the new value in the atomic, and compare it to our original `x`. This still works because the `compare_exchange_strong` compares as-if using `memcmp`, so we are comparing the object representation of `x` against the object representation of the copy of `y` stored in `p`: if the pointers have the same object representation then they have the same value representation, so must be the same and point to the same object.